

Simulazione di un Sistema a N-corpi

Montroni Gregorio e Pasini Alen

28/08/2026

Il progetto

La *struct* e le *class*

Il progetto che abbiamo realizzato con lo scopo di simulare sistemi a N-corpi si fonda su una *struct* e due *class*, rispettivamente *Vector*, *Body* e *Universe*.

La *struct Vector* è stata implementata per gestire le grandezze vettoriali all'interno della simulazione. Poiché il nostro progetto descrive simulazioni in due dimensioni, la *struct* possiede due variabili membro x e y i cui valori sono forniti durante la creazione degli oggetti. Sono stati inoltre compiuti gli *overload* degli operatori $==$, $+$ e $-$ in modo che fosse possibile compiere operazioni tra grandezze vettoriali. Sono state poi implementate funzioni che ritornassero le componenti x e y del vettore e che ne calcolassero la norma.

La *class Body* rappresenta i singoli corpi. Le variabili membro rappresentano le grandezze che caratterizzano un corpo nello spazio, tra cui posizione, velocità, accelerazione, massa e raggio. Queste vengono fornite in costruzione. Attraverso la classe *Body* è anche possibile aggiornare la posizione e la velocità del corpo in seguito a uno *step* di simulazione. È stato inoltre implementato l'*overload* dell'operatore $==$, in modo che due corpi siano considerati uguali se possiedono uno stesso *id* che ottengono una volta inseriti all'interno dell'Universo. Ogni oggetto *Body* contiene poi un oggetto *VertexArray* che rappresenta la scia lasciata dal corpo sullo schermo durante la simulazione.

La *class Universe* è la principale classe del programma. Per poter costruire un oggetto di tale classe è necessario fornire il valore di G , la lunghezza di un intervallo di simulazione dt e il valore di scala che permette di disegnare i corpi sullo schermo. Lo scopo più importante della *class Universe* è il calcolo dell'accelerazione dei singoli corpi dopo un intervallo di tempo, calcolo che non poteva essere svolto all'interno di *Body*. Per poterlo eseguire, infatti, è necessario dover accedere alle posizioni di ciascun corpo. È stato quindi deciso di creare all'interno della classe un vettore di *Body* contenente tutti i corpi della simulazione. In questo modo, attraverso specifiche funzioni sia della *class Universe* che della stessa *class Body*, diventa possibile accedere contemporaneamente a tutti i corpi e alle rispettive posizioni. Sempre nella *class Universe* sono state anche implementate le funzioni per l'aggiornamento delle grandezze dell'intera simulazione (energia, quantità di moto e momento angolare) e per il calcolo complessivo di uno *step* simulativo. In un oggetto *Universe* è inoltre presente un vettore di oggetti *CircleShape* della libreria SFML che contiene la rappresentazione grafica di ciascun corpo.

Il calcolo delle grandezze

Le grandezze gestite dal progetto sono quelle principali della meccanica classica. Ciascun oggetto *Body* possiede un *Vector* posizione, un *Vector* velocità e due *Vector* accelerazione. I vettori relativi all'accelerazione sono due in quanto per il calcolo della velocità dopo un intervallo di tempo dt è necessario avere sia l'accelerazione passata che quelle appena aggiornata. Come già accennato, il calcolo della nuova posizione e della nuova velocità vengono gestite da *Body*, mentre quello dell'accelerazione da *Universe*.

All'interno della *class Universe* sono poi gestite le grandezze relative all'intero Universo, ossia l'energia cinetica, l'energia potenziale, l'energia totale, la quantità di moto e il momento angolare. Attraverso l'interfaccia grafica vengono stampati a schermo i valori iniziali e correnti delle grandezze. Grazie a specifiche configurazioni e tramite i test è stato possibile verificare che in un sistema senza urti si conservano quasi perfettamente l'energia totale, la quantità di moto e il momento angolare.

Nel caso di simulazioni con collisioni, che sono state considerate come urti anelastici, si è verificata la conservazione della quantità di moto.

Affinché fosse visualizzabile sullo schermo che le grandezze si stessero conservando, abbiamo deciso di stampare ad ogni frame la differenza tra le grandezze iniziali e attuali, dividendo il risultato per un valore che rendesse l'errore "normalizzato" rispetto alla specifica configurazione. Abbiamo infatti notato che in simulazioni con valori "realistici" la sola differenza tra le condizioni finali e iniziali poteva trarre in inganno, presentandosi come un numero molto grande. In realtà, paragonandola con i valori iniziali essa risultava essere sempre più che trascurabile. Abbiamo quindi, in un primo momento, deciso di dividere la differenza per il valore iniziale della grandezza, il che ha però causato problemi nel caso in cui esso fosse zero. Abbiamo quindi introdotto degli specifici valori che rappresentassero la "scala" della grandezza nella simulazione e abbiamo diviso le differenze per tali valori. Le formule utilizzate sono state riportate successivamente:

$$\Delta E = \frac{E - E_0}{E_0} \quad \Delta P = \frac{|\vec{P} - \vec{P}_0|}{m |\vec{v}|} \quad \Delta L = \frac{L - L_0}{m |\vec{v}| |\vec{r}|}$$

Insieme alle già citate, G è una delle grandezze più importanti in un sistema gravitazionale di N-corpi. Durante lo sviluppo del nostro programma abbiamo deciso di non utilizzare sempre il valore reale di $G = 6.67 \cdot 10^{-11} \text{ m}^3 \text{ kg}^{-1} \text{ s}^{-2}$ per le simulazioni. In molti casi, infatti, è risultato essere più comodo utilizzare un valore di $G = 1$ in simulazioni con corpi aventi massa dell'ordine di 10^0 . Questa scelta è stata fatta ad esempio nell'implementazione del "Figure-8". Per simili motivi di comodità computazionale abbiamo optato per utilizzare un intervallo di tempo molto maggiore di quello di $dt = 0.005 \text{ s}$ quando abbiamo simulato configurazioni con valori "realistici". Questo era necessario in quanto i tempi di orbita dei pianeti sono troppo grandi per poter essere apprezzati a partire da incrementi dell'ordine del millesimo di secondo.

Le collisioni

Per rendere il più realistico possibile il nostro progetto abbiamo implementato la funzione `check_collision()` in modo da gestire le collisioni tra i corpi nel caso in cui essi abbiano distanza minore della somma dei rispettivi raggi.

La funzione prevede che quando avviene una collisione, i due corpi si fondano come se subissero un urto anelastico. In questo modo la nuova massa e il nuovo raggio sono dati dalla somma dei valori di partenza dei corpi interessati. Per quanto riguarda la posizione e la velocità del nuovo corpo, invece, abbiamo utilizzato le seguenti formule ottenute dalla meccanica classica:

$$\vec{r}_{new} = \frac{m_i \vec{r}_i + m_j \vec{r}_j}{m_i + m_j} \quad \vec{v}_{new} = \frac{m_i \vec{v}_i + m_j \vec{v}_j}{m_i + m_j}$$

dove il nuovo $\vec{r}$ è il centro di massa tra i due corpi e la nuova $\vec{v}$ è la sua velocità, che si conserva prima e dopo l'urto.

Per implementare questo processo abbiamo utilizzato un doppio ciclo per controllare tutti i corpi a due a due e verificare se effettivamente fosse avvenuta una collisione. I due cicli iterano rispettivamente da $i = 0$ e $j = i + 1$ fino al numero totale di corpi presente nell'Universo. Ogni corpo i viene così confrontato con tutti i corpi con indice tra $i + 1$ e l'ultimo corpo del vettore che li contiene.

Nel caso in cui si verifichi una collisione, si memorizzano gli indici i e j . Viene quindi cancellato il secondo corpo e il programma esce dal secondo ciclo attraverso il comando `break`. Lo stesso viene fatto per il primo corpo e con il primo ciclo. Abbiamo deciso di uscire dai cicli in quanto in seguito alla rimozione dei corpi gli `id` slittati non permetterebbero un efficiente confronto tra i corpi. Cancellati i due corpi si va a creare il nuovo corpo con i valori ottenuti dalle precedenti formule e lo si aggiunge all'Universo.

Essendo che non è garantito che ad ogni step del programma si verifichi una sola collisione è necessario utilizzare un ciclo *while* e una variabile di controllo *n*. Se questo non si facesse sarebbe possibile saltare il controllo tra due corpi in seguito all'interruzione dei cicli tramite *break*. Di conseguenza abbiamo fatto in modo che se si verifica una collisione ($n = 1$) i due cicli *for* che gestiscono i confronti tra i corpi vengono svolti nuovamente, questa volta comprendendo anche il neo-formato corpo. Questo viene ripetuto finché non si verificano più collisioni e *n* rimane uguale a 0 per tutte le iterazioni.

Una volta verificata una collisione abbiamo utilizzato due cicli *for* per andare a correggere gli *id* dei corpi che risultano slittati in seguito alla rimozione dei due *Body* che hanno subito l'urto. Svolto anche questo la funzione termina con il calcolo delle accelerazioni dei corpi rimanenti.

La grafica e il file *Configurations.txt*

L'implementazione grafica del progetto è stata eseguita tramite la libreria SFML.

La prima azione che viene svolta dal programma è la creazione di una *window* su cui disegnare gli oggetti. Per facilitare la visualizzazione, abbiamo settato la *window* in modo da avere il punto di coordinate (0,0) al centro dello schermo.

Una volta lanciato il programma l'interfaccia grafica si presenta come uno schermo nero sul quale sono presenti alcune istruzioni e la configurazione di default dell'Universo, un sistema a 3 corpi noto come "Figure-8". Insieme alle istruzioni vengono stampate a schermo le caratteristiche iniziali e correnti dell'Universo, che vengono aggiornate ad ogni frame.

Per passare ad un'altra configurazione si utilizzano i tasti presenti sulla sinistra dello schermo, rappresentati tramite oggetti *RectangleShape*. Premendone uno il programma crea la nuova configurazione leggendola dal file *Configurations.txt*. Nel file sono presenti i parametri che permettono di creare un nuovo oggetto *Universe* e i rispettivi nuovi *Body*. I singoli *Body* sono rappresentati a schermo tramite oggetti *CircleShape* della libreria SFML e se ne possono rappresentare fino a 6 di colore diverso prima che i colori si ripetano.

Affinché il comando di lettura del file *Configurations.txt* avvenga premendo un tasto, abbiamo preso alcuni accorgimenti. Il principale è quello che permette di memorizzare in un vettore la posizione *x* e *y* del mouse quando il tasto sinistro viene premuto e di convertire i valori ottenuti in modo che siano coerenti con la scelta di posizionare il punto (0,0) al centro dello schermo. Di default, infatti, SFML riconosce la posizione del mouse come (0,0) quando questo è nell'angolo in alto a sinistra dello schermo. Una volta memorizzata la posizione il programma controlla se essa sia contenuta all'interno dei confini del rettangolo che rappresenta il bottone. Se questo si verifica, la nuova configurazione viene letta dal file.

La lettura dal file avviene attraverso una funzione che prende in input una stringa con il nome della configurazione e ritorna un vettore di *double* contenente i parametri del nuovo Universo e dei nuovi *Body*. La funzione, una volta aperto il file, lo legge riga per riga fino a trovare quella con il titolo della configurazione. Salva quindi tutti i successivi valori numerici all'interno di un vettore fino a quando non incontra la stringa "fine". A quel punto smette di leggere e ritorna il vettore, che viene poi utilizzato dal programma per creare la nuova configurazione. Per l'implementazione della funzione di lettura sono stati utilizzati alcuni costrutti della Libreria Standard appartenenti ai file di intestazione "*fstream*" e "*sstream*" che non sono stati affrontati a lezione.

Poiché le distanze tra i corpi e le loro stesse dimensioni possono variare grandemente in base alla configurazione, durante la creazione dell'oggetto *Universe* viene fornito un valore di scala. Questo viene utilizzato in modo da poter rappresentare in maniera sempre opportuna i corpi sullo schermo.

Sempre tramite la libreria SFML vengono stampati a schermo i valori delle grandezze dell'Universo, che vengono aggiornate ad ogni frame.

In generale, sia per i corpi che per i tasti che per le legende abbiamo scelto di creare dei vettori che contenessero oggetti dello stesso tipo presi dalla libreria SFML. In questo modo ad ogni frame vengono svolti diversi cicli *for* della lunghezza dei rispettivi vettori che permettono di stampare a schermo gli oggetti in essi contenuti. Diversi costrutti utilizzati nell'implementazione grafica del progetto non sono stati affrontati direttamente a lezione ma approfonditi da noi autonomamente durante lo sviluppo del programma.

Le configurazioni

Le configurazioni che abbiamo reso visualizzabili all'interno del nostro programma sono 7 e ciascuna è stata scelta in quanto mostra una particolare caratteristica del nostro progetto.

Le configurazioni “*Figure-8*” e “*Henon Configuration*” rappresentano configurazioni che, nonostante le approssimazioni dovute dall'algoritmo utilizzato, rimangono stabili nel tempo. Si può quindi apprezzare la conservazione dell'energia, della quantità di moto e del momento angolare. I dati per la prima configurazione sono stati forniti come consegna al compito, mentre quelli per la seconda sono stati ottenuti dal sito internet di simulazione *trisolarchaos.com*.

Le configurazioni “*Pulsating Hexagon*”, “*Yarn Configuration*” e “*The Chase*” sono invece esemplificative in quanto con il passare del tempo diventano instabili e i corpi collidono. Si può quindi osservare la conservazione della quantità di moto prima e dopo l'urto e la non conservazione dell'energia e del momento angolare. Le prime due configurazioni sono state ottenute dal sito *trisolarchaos.com*, mentre i dati per la terza sono stati forniti dall'Intelligenza Artificiale Claude AI. In generale, per tutte le precedenti cinque configurazioni è stato utilizzato un valore di $G = 1$ e $dt = 0.005$ s.

Per ultime, le configurazioni “*Earth & Moon*” e “*Inner Solar System*” rappresentano la possibilità di simulare all'interno del nostro programma sistemi planetari realmente esistenti. I parametri di posizione, velocità e massa utilizzati, infatti, sono quelli reali dei corpi celesti. L'unico dato ad essere stato manipolato è il raggio dei corpi. Rimanendo coerenti, infatti, sarebbe stato impossibile visualizzare i corpi, in quanto le loro dimensioni sono troppo piccole rispetto alle distanze che li separano. Abbiamo perciò deciso di moltiplicare per 14 il raggio solare e per 614 quello dei pianeti. Abbiamo inoltre variato il valore di dt in maniera che fosse apprezzabile il movimento dei corpi.

Funzionamento di un'intera simulazione

Immaginando di essere appena passati ad una nuova configurazione, il complessivo funzionamento del programma può essere riassunto nei seguenti passaggi:

1. Reset dell'Universo
 - L'oggetto *Universe* già esistente viene resettato. Vengono riportate a zero tutte le grandezze e vengono cancellati tutti gli oggetti *Body* e tutti i *CircleShape*. Vengono inseriti i nuovi valori di G , dt e $scale$ letti in input dal file *Configuration.txt*.
2. Creazione dei nuovi corpi
 - Il programma legge in input i parametri dei nuovi corpi, i quali vengono aggiunti al vettore nell'oggetto *Universe*. Allo stesso modo vengono creati i nuovi *CircleShape*.
3. Simulazione di uno step temporale
 - Viene settato il valore di dt in modo che sia uguale per tutti i corpi. Nel caso si stia procedendo manualmente all'indietro, infatti, dt deve essere negativo.
 - Viene controllato se si sono verificate delle collisioni.
 - Vengono aggiornati per ciascun corpo la posizione, l'accelerazione e la velocità, esattamente in questo ordine. Vengono quindi calcolati i nuovi valori delle variabili dell'Universo. L'accelerazione passata viene resa uguale a quella aggiornata, in modo che sia poi utilizzabile alla successiva iterazione.

4. Aggiornamento dello schermo
 - Ad ogni frame vengono ridisegnati i corpi nella posizione corretta. Allo stesso tempo vengono aggiornati i valori delle grandezze dell'Universo e le informazioni sulla simulazione.
5. Ripetizione del ciclo
 - Nel caso in cui non venga premuto alcun tasto, il programma ripete gli step 3 e 4. Nel caso in cui venga caricata una nuova configurazione, si riparte dal primo.

Utilizzo del programma

Una volta lanciato il programma, sullo schermo compare la configurazione planetaria di default, ossia la "8-Figure". In principio la simulazione è ferma e può essere attivata premendo lo spazio. Se invece si vuole procedere avanti o indietro manualmente è possibile farlo utilizzando le frecce sulla tastiera. Una volta che la simulazione ha inizio, la si può interrompere premendo di nuovo lo spazio. Che la simulazione sia in corso o che essa sia ferma è possibile ritornare alla posizione iniziale premendo i tasti `alt + R`.

In alto a destra nello schermo è possibile visualizzare alcune caratteristiche generali della simulazione, ossia il numero di step eseguiti, i valori di G e di dt e il numero di corpi presenti nella simulazione. Sul lato sinistro sono invece presenti i valori delle grandezze dell'Universo e le differenze tra i valori iniziali e quelli correnti.

Sono infine presenti, sempre a sinistra, i tasti che permettono di spostarsi tra una configurazione e l'altra. Il tasto relativo alla configurazione in atto è colorato di giallo, mentre i restanti in blu. Premendo un generico tasto si passa alla configurazione indicata, la quale comincia dalle condizioni iniziali presenti nel file *Configuration.txt*.

I test

Per verificare la correttezza del nostro codice abbiamo implementato diversi test. In generale, la nostra strategia è stata di scrivere test sia prima che dopo l'implementazione delle principali funzionalità. Lo scrivere prima ci ha permesso di immaginare a grandi linee come la funzionalità dovesse essere, mentre lo scrivere dopo ci ha permesso di testarla in maniera approfondita. Inoltre, poiché il codice andava arricchendosi durante lo sviluppo del programma, è stato necessario aggiornare continuamente i test in modo che rimanessero corretti e funzionali.

Per la scrittura dei test relativi a delle intere simulazioni è stata utilizzata l'Intelligenza Artificiale Claude AI. In particolare, fornendo nel prompt la configurazione studiata, abbiamo richiesto i valori delle grandezze dopo un certo numero di iterazioni. Siamo quindi riusciti a testare che le funzioni di calcolo di posizione, accelerazione e velocità funzionassero adeguatamente così come il calcolo delle grandezze dell'Universo. Allo stesso modo abbiamo verificato le varie conservazioni.

Utilizzo di sistemi di Intelligenza Artificiale

Come già riportato in precedenza abbiamo utilizzato sistemi di Intelligenza Artificiale in alcune parti del nostro progetto. L'uso principale è stato per la generazione dei test, in modo che i valori delle grandezze studiate fossero i più precisi possibile. Per lo stesso motivo abbiamo generato con l'Intelligenza Artificiale i valori iniziali di alcune configurazioni così da ottenere delle simulazioni interessanti da osservare.

È stata infine utilizzata l'Intelligenza Artificiale per la spiegazione di alcuni passaggi necessari per l'implementazione grafica del progetto e per la lettura di valori da un file. Per fare questo abbiamo generalmente richiesto all'Intelligenza Artificiale degli esempi generici e non strettamente

applicabili al nostro programma. In questo modo è risultata fondamentale la nostra comprensione e il nostro intervento diretto sul codice scritto.

Note conclusive

Per lo sviluppo del programma è stata utilizzata la piattaforma GitHub.

Per eseguire il programma, dopo aver svolto la compilazione si può lanciare l'eseguibile con nome *n_bodies_simulation* contenuto nella directory *Release* che si trova a sua volta nella directory *build*.